

IF2211: Laporan Tugas Besar 3

**Pemanfaatan Algoritma Pattern Matching pada Chromium
Browser Extension untuk Deteksi Judi Online**



Dipersiapkan oleh NekoGarong:

13524003 Faiq Azzam Nafidz

13524018 Muhammad Nafis Habibi

13524110 Jennifer Khang

**PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL. GANESA 10, BANDUNG 40132
2025**

Daftar Isi

Bab I: Deskripsi Tugas.....	2
Bab II: Landasan Teori.....	4
2.1. String/Pattern Matching.....	4
2.2. Algoritma Knuth-Morris-Pratt.....	4
2.3. Algoritma Boyer-Moore.....	5
2.4. Regular Expression.....	6
2.5. Algoritma Aho-Corasick.....	7
2.6. Algoritma Rabin-Karp.....	8
2.7. Weighted Levenshtein Distance.....	9
2.8. Chromium Browser Extension.....	9
2.9. DOM Traversal.....	10
Bab III: Analisis Pemecahan Masalah.....	12
3.1. Text Node DOM.....	12
3.2. Keyword Judi Online.....	12
3.3. Pemetaan Masalah ke Elemen Pattern Matching.....	12
3.4. Arsitektur Extension.....	13
Bab IV: Implementasi dan Pengujian Program.....	15
4.1. Implementasi Program.....	15
4.1.1. Struktur Folder.....	15
4.1.3. Alur Program.....	16
4.1.4. Struktur Data.....	18
4.2. Pengujian Program.....	21
4.2.1. Faktor yang Diujikan.....	21
4.2.2. Hasil Pengujian.....	21
Bab V: Kesimpulan dan Saran.....	37
5.1. Kesimpulan.....	37
5.2. Saran.....	37
Lampiran.....	39
Daftar Pustaka.....	41

Bab I: Deskripsi Tugas

Penyebaran konten judi *online* (judol) di internet merupakan permasalahan yang hingga saat ini masih ada di Indonesia. Konten tersebut tidak hanya muncul pada situs-situs ilegal yang berdiri sendiri, tetapi juga menyusup ke berbagai halaman web umum dalam bentuk iklan banner, tautan tersembunyi, kolom komentar, hingga konten yang disisipkan secara terprogram ke dalam halaman yang tampak valid. Fenomena ini diperparah dengan meningkatnya aksesibilitas internet di kalangan masyarakat luas, termasuk kelompok rentan seperti pelajar, mahasiswa, dan pengguna yang belum memiliki literasi digital yang memadai, sehingga risiko paparan terhadap konten berbahaya ini pun semakin besar.

Yang menjadikan permasalahan ini semakin kompleks adalah strategi penyamaran yang digunakan oleh pelaku penyebaran konten judi online. Konten tersebut seringkali ditulis dengan variasi ejaan yang tidak biasa, seperti penggantian huruf dengan angka, penyisipan karakter khusus, pemisahan kata, atau penggunaan bahasa gaul dan akronim yang terus berkembang. Akibatnya, sistem filter konvensional yang umumnya tidak dapat menghadapi strategi tersebut karena hanya mengandalkan pencocokan kata kunci secara eksak (*exact matching*). Di sisi lain, pemblokiran manual oleh pengguna maupun moderator konten membutuhkan waktu dan sumber daya yang tidak sedikit, dan seringkali tidak mampu mengikuti laju penyebaran konten yang begitu cepat. Kondisi ini menunjukkan perlunya pendekatan deteksi yang lebih adaptif, otomatis, dan mampu mengenali pola teks secara fleksibel, termasuk terhadap variasi penulisan yang sengaja disamarkan.



Gambar 1: Visualisasi Hasil Proyek

Untuk mengatasi permasalahan tersebut, perlu adanya sebuah detector judi online yang strategis. Pada Tugas Besar 3 IF2211 Strategi Algoritma, tim harus membuat sebuah chromium browser extension bernama

Judol Detector menggunakan TypeScript untuk mendeteksi konten judi online pada halaman web. Extension ini dirancang untuk mengenali pola teks yang sering digunakan pada situs judi online. Dalam implementasinya, extension wajib mengenali pola teks menggunakan algoritma Knuth-Morris-Pratt (KMP), Boyer-Moore (BM), Regular Expression (RegEx), serta fuzzy matching menggunakan Weighted Levenshtein Distance.

Bab II: Landasan Teori

2.1. String/Pattern Matching

Pattern matching atau pencocokan string merupakan proses menemukan pola teks yang sesuai di dalam teks yang lebih panjang. Jika diberikan sebuah teks T yang merupakan string sepanjang n karakter, dan sebuah pattern P yang merupakan string sepanjang m karakter dengan asumsi $m < n$, maka tujuan dari pencocokan string adalah menemukan lokasi atau indeks dalam teks T di mana pattern P pertama kali muncul. Pencocokan string mencari semua indeks i sehingga $T[i..i + m - 1] = P$. Pattern Matching dibagi menjadi dua kelompok sebagai berikut.

- a. **Algoritma Pencocokan String yang Tepat (Exact Matching):** Kelompok algoritma yang menemukan pola eksak tanpa kesalahan. Contoh: Knuth–Morris–Pratt (KMP), Rabin–Karp, Boyer–Moore, dan Aho–Corasick.
- b. **Algoritma Pencocokan String Perkiraan (Fuzzy Matching):** Kelompok algoritma yang mengizinkan sejumlah kesalahan berdasarkan *threshold* tertentu (penyisipan, penghapusan, substitusi, atau transposisi). Contoh: Levenshtein.

Terdapat beberapa pendekatan dalam menyelesaikan permasalahan pencocokan string, yang dibedakan berdasarkan strategi pergeseran pattern terhadap teks. Pada Tugas Besar 3 IF2211 Strategi Algoritma, tim akan menggunakan KMP, Boyer-Moore, Aho-Corasick, dan Rabin-karp sebagai algoritma pencocokan string yang tepat, serta Levenshtein sebagai algoritma pencocokan string perkiraan, dalam proses pattern matching untuk membangun Judi Online Detector.

2.2. Algoritma Knuth-Morris-Pratt

Algoritma Knuth-Morris-Pratt (KMP) bekerja dengan cara memindai pattern dari kiri ke kanan. Ketika terjadi ketidakcocokan (mismatch) antara pattern P dan teks T , algoritma KMP tidak mengulang perbandingan yang telah dilakukan sebelumnya, melainkan memanfaatkan informasi dari karakter-karakter yang sudah dicocokkan untuk menentukan seberapa jauh pattern harus digeser.

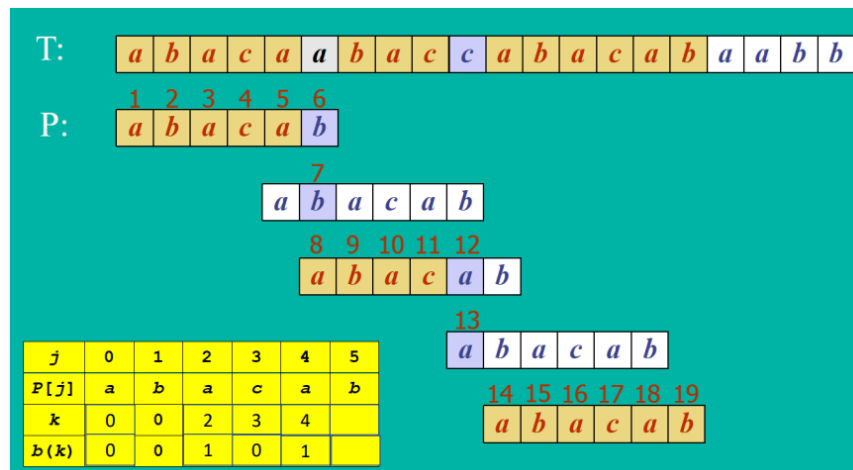
Berikut adalah beberapa tahapan dari algoritma KMP.

- a. **Fase Pra-pemrosesan Pola:** Kunci dari algoritma KMP adalah fungsi pinggiran (border function atau failure function), yang dinotasikan sebagai $b(k)$. Fungsi ini didefinisikan sebagai ukuran terbesar dari prefix $P[0..k]$ yang sekaligus merupakan

suffix dari $P[1..k]$. Fungsi pinggiran ini dikomputasi secara terpisah dalam tahap ini sebelum proses pencarian dimulai, dengan kompleksitas waktu $O(m)$.

- b. **Fase Pencarian:** Apabila terjadi mismatch pada posisi j dalam pattern, nilai $b(j-1)$ digunakan untuk menentukan posisi pergeseran baru, sehingga perbandingan karakter dapat dilanjutkan dari posisi yang tepat tanpa harus kembali ke awal. Proses berakhir ketika indeks tes mencapai akhir, sehingga seluruh teks sudah diperiksa.

Kompleksitas waktu keseluruhan algoritma KMP adalah $O(m + n)$, di mana $O(m)$ digunakan untuk menghitung fungsi pinggiran dan $O(n)$ untuk proses pencarian. Keunggulan utama algoritma ini adalah bahwa pointer pada teks T tidak pernah bergerak mundur. Namun, KMP memiliki kelemahan berada pada konstanta waktu yang lebih besar dibanding algoritma Boyer–Moore ketika alfabet berukuran besar.



Gambar 2: Ilustrasi Pencocokan menggunakan KMP

(Sumber: Situs Rinaldi Munir)

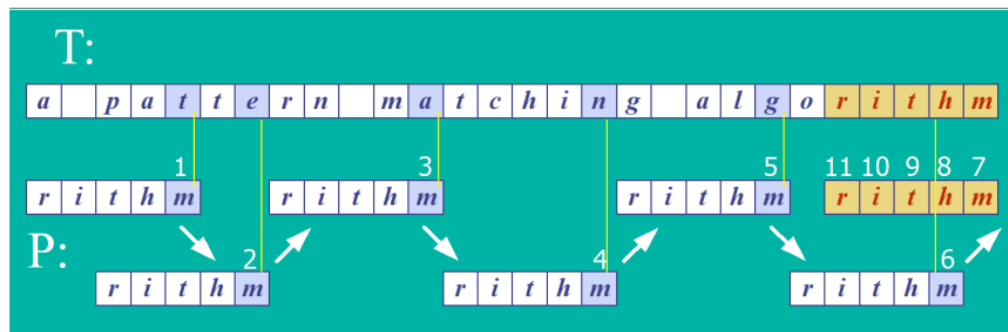
2.3. Algoritma Boyer-Moore

Algoritma Boyer-Moore memindai karakter pattern dari kanan ke kiri (*looking-glass technique*), dimulai dari karakter terakhir pattern. Pendekatan ini memungkinkan Boyer-Moore untuk melewati lebih banyak karakter teks sekaligus, menggunakan heuristik Bad Character dan Good Suffix, sehingga dalam banyak kasus praktis menjadi jauh lebih cepat dari algoritma lainnya.

Ketika terjadi mismatch antara $T[i]$ dan $P[j]$ dengan karakter $T[i] = x$, terdapat tiga kemungkinan kasus yang ditangani secara berurutan.

- Last Occurrence (Bad Character):** Jika karakter teks x yang menyebabkan mismatch juga terdapat pada pola, maka pola akan digeser ke kanan hingga kemunculan terakhir karakter x .
- Shift Satu Posisi:** Apabila karakter x memang terdapat pada pola tetapi posisinya berada di sebelah kanan titik mismatch, maka nilai pergeseran dari heuristik Bad Character menjadi negatif dan digeser sebanyak satu kali.
- Full Shift:** Jika karakter x sama sekali tidak muncul pada pola, maka pola dapat langsung digeser sehingga karakter pertama pola berada tepat setelah posisi x pada teks.

Boyer-Moore dibagi menjadi tahapan preprocess dan pencarian. Preprocess akan membangun tabel Last Occurrence Table $L(x)$ untuk setiap karakter alfabet dan tabel Good Suffix untuk menentukan pergeseran minimal saat sufiks cocok. Pencarian akan berakhir ketika sudah melewati batas teks. Kompleksitas waktu kasus terburuk Boyer-Moore adalah $O(nm + |A|)$, di mana $|A|$ adalah ukuran alfabet yang digunakan. Namun dalam praktiknya, algoritma ini sangat efisien ketika alfabet yang digunakan berukuran besar. Sebaliknya, Boyer-Moore kurang efisien untuk teks biner atau alfabet berukuran kecil.



Gambar 3: Ilustrasi Pencocokan menggunakan Boyer-Moore

(Sumber: Situs Rinaldi Munir)

2.4. Regular Expression

Regular expression (Regex) adalah notasi formal yang digunakan untuk mendefinisikan pola (pattern) pencarian dalam sebuah string. Berbeda dengan KMP dan Boyer-Moore yang mencocokkan string secara eksak, Regex memungkinkan pendefinisian pola yang bersifat fleksibel dan generatif, sehingga mampu mencocokkan berbagai variasi bentuk teks sekaligus dalam satu ekspresi. Notasi dasar Regex mencakup sejumlah metacharacter yang memiliki makna khusus.

Special character	Meaning
.	matches any element
^	matches beginning of sequence
\$	matches end of sequence
*	zero or more repetitions
+	one or more repetitions
?	zero or one repetition
{ <i>m</i> }	exactly <i>m</i> repetitions
{ <i>m</i> , <i>n</i> }	between <i>m</i> and <i>n</i> repetitions
	matches either regex before or after the symbol

Gambar 4: Tabel Regex

(Sumber: https://miro.medium.com/v2/resize:fit:850/1*xAjjtY1N1CWCVhk3mpkXuww.png)

Pola regex umumnya dibangun dari empat kategori elemen:

1. Literal character yang mewakili diri-nya sendiri.
2. Metacharacter, yakni simbol abstrak untuk menyatakan kelas karakter, jangkar posisi, atau operator kuantifikasi.
3. Quantifier, yang menentukan kardinalitas kemunculan bagian pola.
4. Grouping dan alternation, yang memungkinkan penyusunan sub-pola bersarang dan pilihan pola alternatif.

2.5. Algoritma Aho-Corasick

Algoritma Aho-Corasick menangani pencarian banyak pola secara serentak dengan membangun sebuah finite-state automaton. Struktur inti automaton mencakup fungsi goto, failure, dan output sehingga seluruh pola (kamus) dapat ditemukan dalam sekali lintasan teks tanpa backtracking. Berikut adalah tahapan dari algoritma ini.

- a. **Konstruksi Trie (Fungsi Goto):** Seluruh pattern dimasukkan satu per satu ke dalam sebuah struktur pohon trie. Setiap simpul dalam trie mewakili sebuah prefix dari salah satu pattern, dan setiap tepi mewakili transisi karakter dari satu simpul ke simpul berikutnya. Simpul-simpul yang merupakan akhir dari suatu pattern ditandai sebagai simpul output.
- b. **Konstruksi Fungsi Kegagalan (Failure Function):** Fungsi kegagalan dibangun menggunakan penelusuran BFS (breadth-first search) pada trie. Untuk setiap simpul *s*, fungsi kegagalan *f(s)* didefinisikan sebagai simpul yang mewakili sufiks terpanjang dari string yang merepresentasikan *s*, yang sekaligus merupakan prefix dari pola. Simpul-simpul anak dari akar yang tidak memiliki transisi langsung kembali ke akar.

- c. **Konstruksi Fungsi Output:** Fungsi output $O(s)$ memetakan setiap simpul s ke himpunan pattern yang berakhir di simpul tersebut. Selain pattern yang secara langsung berakhir di s , fungsi output juga menyertakan pattern yang ditemukan melalui rantai fungsi kegagalan, yaitu $O(s) = O(s) \cup O(f(s))$.
- d. **Tahap Pencarian:** Teks T dibaca karakter demi karakter. Pada setiap langkah, automaton berpindah ke simpul berikutnya mengikuti fungsi goto. Apabila tidak ada transisi yang valid untuk karakter saat ini, automaton mengikuti fungsi kegagalan hingga ditemukan transisi yang valid atau kembali ke akar. Setiap kali simpul yang dikunjungi memiliki output yang tidak kosong, seluruh pattern dalam himpunan output tersebut dicatat sebagai kemunculan.

Kompleksitas waktu preprocessing untuk membangun trie, fungsi kegagalan, dan fungsi output adalah $O(M \times |A|)$, di mana M adalah total panjang seluruh pattern dan $|A|$ adalah ukuran alfabet. Tahap pencarian memiliki kompleksitas $O(n + z)$, di mana n adalah panjang teks dan z adalah jumlah total kemunculan seluruh pattern yang ditemukan.

2.6. Algoritma Rabin-Karp

Algoritma Rabin-Karp memanfaatkan konsep hashing untuk membandingkan pattern dengan substring teks secara efisien, alih-alih membandingkan karakter satu per satu seperti yang dilakukan KMP maupun Boyer-Moore. Secara algoritmik, tahapan Rabin-Karp dapat dibagi menjadi beberapa tahapan.

- a. **Pra-pemrosesan Hash Awal:** Fungsi hash polinomial berbasis bilangan prima dikomputasi terlebih dahulu dan digunakan untuk mengonversi string menjadi nilai numerik.
- b. **Pembaruan Rolling Hash:** Setiap kali jendela digeser satu posisi ke kanan, nilai hash diperbarui dengan menghapus kontribusi karakter yang keluar dan menambahkan kontribusi karakter baru yang masuk, sedemikian sehingga setiap pembaruan hanya membutuhkan waktu konstan $O(1)$.
- c. **Pencocokan dan Verifikasi:** Pada setiap posisi i dalam teks, nilai hash substring $T[i..i+m-1]$ dibandingkan dengan nilai hash pattern. Jika keduanya sama, dilakukan verifikasi karakter per karakter untuk mengonfirmasi kecocokan yang sebenarnya. Jika berbeda, jendela digeser dan rolling hash diperbarui. Proses ini berlanjut hingga seluruh teks habis dipindai.

Kompleksitas waktu rata-rata algoritma Rabin-Karp adalah $O(m + n)$, dengan $O(m)$ untuk komputasi hash awal dan $O(n)$ untuk penelusuran teks. Namun, ketika terjadi banyak tabrakan hash, kompleksitasnya menjadi $O(mn)$.

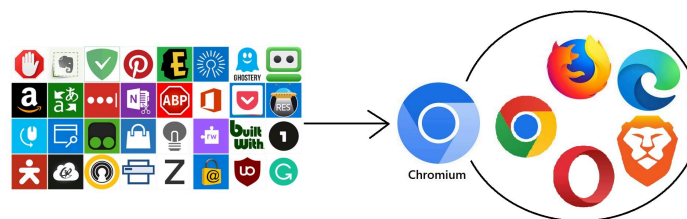
2.7. Weighted Levenshtein Distance

Levenshtein Distance atau jarak edit adalah ukuran kesamaan antara dua string yang dihitung berdasarkan jumlah operasi minimum yang diperlukan untuk mengubah satu string menjadi string lainnya. Operasi yang diizinkan mencakup tiga jenis, yakni penyisipan (insertion) satu karakter, penghapusan (deletion) satu karakter, dan penggantian (substitution) satu karakter dengan karakter lain. Semakin kecil nilai jarak Levenshtein antara dua string, semakin mirip kedua string tersebut. Konsep ini menjadi dasar dari teknik fuzzy matching.

Dalam varian Weighted Levenshtein Distance, setiap jenis operasi diberikan bobot (weight) yang berbeda-beda sesuai dengan konteks permasalahan. Hal ini memungkinkan sistem untuk memperlakukan jenis kesalahan tertentu sebagai lebih "mahal" atau lebih "murah" dibandingkan jenis kesalahan lainnya. Komputasi jarak ini umumnya dilakukan menggunakan pendekatan pemrograman dinamis dengan matriks berukuran $(m+1) \times (n+1)$, di mana m dan n adalah panjang kedua string yang dibandingkan, sehingga kompleksitas waktunya adalah $O(mn)$.

2.8. Chromium Browser Extension

Browser extension atau ekstensi peramban adalah program komputer kecil yang memperluas fungsionalitas web. Pada ekosistem peramban berbasis Chromium, ekstensi dikembangkan menggunakan teknologi web standar, yaitu HTML, CSS, JavaScript, dan TypeScript. Setiap ekstensi dideskripsikan melalui berkas konfigurasi bernama `manifest.json` yang mendefinisikan metadata ekstensi, *permissions*, serta komponen-komponen yang digunakan.



Gambar 5: *Environment* Ekstensi Chromium

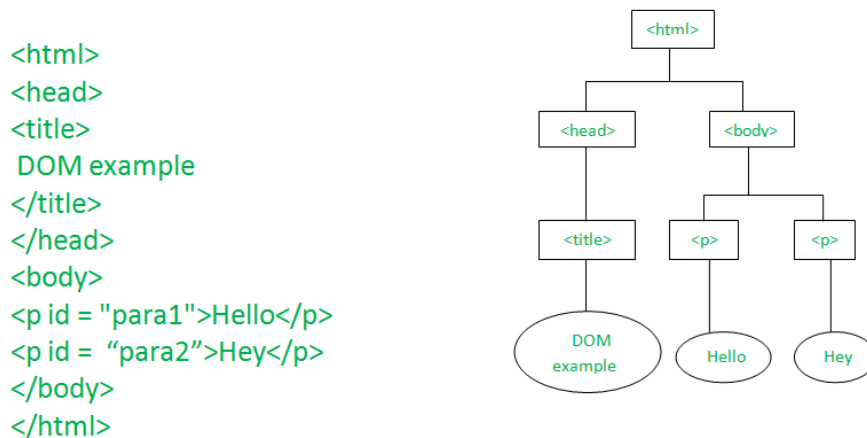
(Sumber: <https://qubika.com/wp-content/uploads/2024/12/chromium-extensions.jpg>)

Arsitektur ekstensi Chromium terdiri dari beberapa komponen utama.

- Background script atau service worker berjalan di latar belakang dan bertanggung jawab untuk mengelola logika utama serta komunikasi antar komponen. Komunikasi antara komponen-komponen ini dilakukan melalui mekanisme message passing menggunakan API chrome.
- Content script adalah skrip yang disuntikkan langsung ke dalam halaman web yang sedang dibuka pengguna, sehingga memiliki akses terhadap DOM (Document Object Model) halaman tersebut.
- Popup merupakan antarmuka pengguna kecil yang muncul ketika ikon ekstensi di toolbar ditekan.

2.9. DOM Traversal

Document Object Model adalah application programming interface (API) lintas platform yang menginterpretasi file HTML atau XML sebagai struktur tree dimana tiap simpul merupakan sebuah objek yang merepresentasikan suatu bagian dari dokumen file tersebut. Metode yang terdapat pada DOM memberikan akses untuk mengubah struktur, presentasi, dan kandungan dokumen.



Gambar 6: Contoh DOM Tree

(Sumber: <https://media.geeksforgeeks.org/wp-content/uploads/DOMExample-3-1.png>)

DOM traversal merujuk pada proses penelusuran pohon DOM untuk mengakses atau memproses simpul-simpul tertentu. Proses ini dapat dilakukan menggunakan berbagai metode yang

disediakan oleh antarmuka DOM. Dalam konteks ekstensi peramban, content script memanfaatkan DOM traversal untuk memindai seluruh konten teks dalam halaman web secara sistematis. Simpul teks (text node) dapat diekstrak menggunakan TreeWalker atau rekursi pada childNodes, kemudian dianalisis menggunakan algoritma pencocokan string untuk mendeteksi keberadaan kata kunci yang tidak dikehendaki. Elemen yang terdeteksi mengandung konten berbahaya selanjutnya dapat dimanipulasi dengan menambahkan atribut style untuk memberikan efek highlight visual kepada pengguna.

Bab III: Analisis Pemecahan Masalah

3.1. Text Node DOM

Pendeteksian bahwa suatu situs mengandung unsur-unsur terkait judi *online* dapat dilihat dari teks yang ada di halamannya. Oleh karena itu, untuk mendeteksi konten judi *online*, perlu diambil terlebih dahulu informasi terkait teks yang ada di dalam halaman terkait. Setiap teks yang ditampilkan pada halaman web disimpan pada *text node* di dalam *DOM*.

Text node adalah salah satu jenis simpul pada *Document Object Model (DOM)* yang memuat teks literal dari halaman web. Pada *HTML file*, *text node* direpresentasikan oleh teks yang diapit oleh *tag* yang merepresentasikan suatu *element node*, seperti `<div>`, `<p>`, `<body>`, dan lain sebagainya. Teks tersebut akan tampil di halaman web sebagai teks sebenarnya dan dimanipulasi dengan *styling* yang diaplikasikan terhadap simpul induk yang mengandungnya.

Pengekstrakan *text node* dari halaman web dilakukan dengan mendapatkan terlebih dahulu *HTML file* dari halaman web tersebut. Setelah itu, buat *DOM* terhadap *HTML file* tersebut (dalam Typescript, *DOM* dapat dibuat dengan pengiterasinya sekaligus menggunakan metode `createTreeWalker`). *DOM* yang terbuat ditelusuri simpulnya satu per satu untuk menemukan setiap *text node* yang ada di dalamnya sehingga nantinya bisa diproses untuk pendeteksian konten judi *online*.

3.2. Keyword Judi Online

Menentukan apakah teks dalam suatu *text node* mengandung unsur judi *online* salah satu caranya dengan mengecek apakah suatu kata kunci (*keyword*) muncul dalam teks tersebut. Pada tugas ini, diberikan daftar kata kunci yang jika suatu *text node* mengandung kata tersebut, *text node* tersebut akan dianggap mengandung unsur judi *online*. Pencarian dan pencocokan teks dalam *text node* dengan setiap kata kunci yang ada dilakukan dengan metode pencocokan eksak (*exact matching*) menggunakan salah satu dari berbagai algoritma, pencocokan menggunakan ekspresi reguler (*regex matching*), atau pencocokan cukup tepat (*fuzzy matching*) yang akan dijelaskan pada bagian Algoritma Penyelesaian Masalah.

3.3. Pemetaan Masalah ke Elemen *Pattern Matching*

Masalah ini, yaitu pada bagian *exact matching*, sebenarnya bisa dipetakan ke dalam persoalan algoritma umum yaitu pencocokan pola dalam suatu teks atau *pattern matching*. Umumnya, persoalan *pattern matching* meminta untuk mencari suatu pola yang ditemukan pertama dalam teks atau keseluruhan pola yang sesuai dalam teks. Dalam tugas ini, dibutuhkan pencarian seluruh pola yang

sesuai dalam teks untuk mendapatkan informasi yang dibutuhkan, seperti kata kunci apa saja yang didapatkan, banyak kemunculan suatu kata kunci, banyak kata kunci yang terdapat dalam teks, dan lain sebagainya.

Dalam *pattern matching*, didefinisikan simbol T (teks), yaitu string dengan panjang n karakter, dan P (*pattern/pola*), yaitu string dengan panjang m karakter (asumsi $m \ll n$) yang akan dicari dalam teks. Permasalahan ini dipetakan dengan menganggap teks dalam suatu *text node* sebagai T dan kata kunci yang ingin dicari di dalam teks sebagai P . Sebagai contoh, T : “Situs paling gacor no rungkad” dan P : “gacor” artinya akan dicari pola “gacor” pada suatu *text node* yang berisi teks sebagaimana di simbol T .

Beberapa algoritma dapat diterapkan untuk menyelesaikan persoalan *pattern matching* ini. Namun sebelum itu, perlu diketahui beberapa konsep string. Misalkan suatu string sepanjang n $S = x_0x_1...x_{n-1}x_n$ (x_i artinya karakter pada string urutan ke- i), maka $S[0..k]$ (string dengan karakter string S dari x_0 sampai x_k) adalah *prefix* dari S . Di sisi lain, $S[k..n-1]$ (string dengan karakter string S dari x_k sampai x_{n-1}) adalah *suffix* dari S . Variabel k tersebut bisa bernilai dari 0 sampai $n-1$.

Algoritma naif yang bisa diterapkan untuk persoalan ini adalah *brute force*, dengan mengecek setiap posisi di T apakah merupakan P dimulai dari posisi tersebut. Algoritma dimulai dengan permulaan P berada di posisi permulaan T . Jika suatu posisi bukan merupakan awalan dari P , maka pencocokan P terhadap T digeser sebanyak 1 posisi. Jika sebagian permulaan P ada di suatu posisi T , cek karakter P selanjutnya apakah posisi selanjutnya di T juga merupakan karakter yang sama. Tentunya algoritma ini akan memakan waktu banyak pada kasus terburuk, yaitu ketika T memiliki banyak awalan P di dalamnya, namun tidak sepenuhnya merupakan P .

Maka dari itu, persoalan ini akan diselesaikan dengan algoritma lain yang sudah ditemukan sebelumnya. Berbagai algoritma yang dipilih untuk menyelesaikan permasalahan *exact matching* ini adalah algoritma **Knuth-Morris-Pratt**, **Boyer Moore**, **Aho-Corasick**, dan **Rabin Karp**. Penjelasan terhadap setiap algoritma tersebut dapat dilihat di bagian Landasan Teori.

3.4. Arsitektur Extension

Permasalahan ini, dalam tugas ini, perlu diimplementasikan pada sebuah *extension browser* berbasis Chromium. Secara umum, ekstensi ini dibangun dengan tiga komponen utama, yaitu lapisan *popup*, *background*, dan *content*.

Lapisan *popup* menjadi antarmuka pengguna dengan ekstensi. Dalam *popup*, pengguna dapat memilih algoritma yang ingin digunakan untuk proses *exact matching*, mengaktifkan/menonaktifkan opsi tambahan (seperti blur pada konten yang terdeteksi mengandung unsur judi *online*), menjalankan *scan* terhadap halaman web yang sedang dibuka, serta menghilangkan anotasi yang dihasilkan oleh proses *scan*. Lapisan ini juga menjadi tempat pengguna dapat melihat statistik *scan* yang dilakukan, mulai dari algoritma yang digunakan, total kecocokan, waktu eksekusi setiap metode pencarian (*exact*, *regex*, dan *fuzzy matching*), total kecocokan setiap metode pencarian, dan perbandingan banyak kemunculan antara setiap kata kunci yang ditemukan.

Lapisan *background* menjadi lapisan penghubung antara antarmuka ekstensi dengan logika ekstensi. Lapisan ini mengatur komunikasi dari lapisan *popup* sebagai permintaan pengguna ke lapisan *content* dan dari lapisan *content* sebagai hasil dari permintaan pengguna ke lapisan *popup*. Hal ini diperlukan untuk mengabstraksi ketergantungan lapisan *popup* dengan *content* sehingga pengembangan kedua lapisan tersebut masing-masing bisa dikerjakan secara paralel.

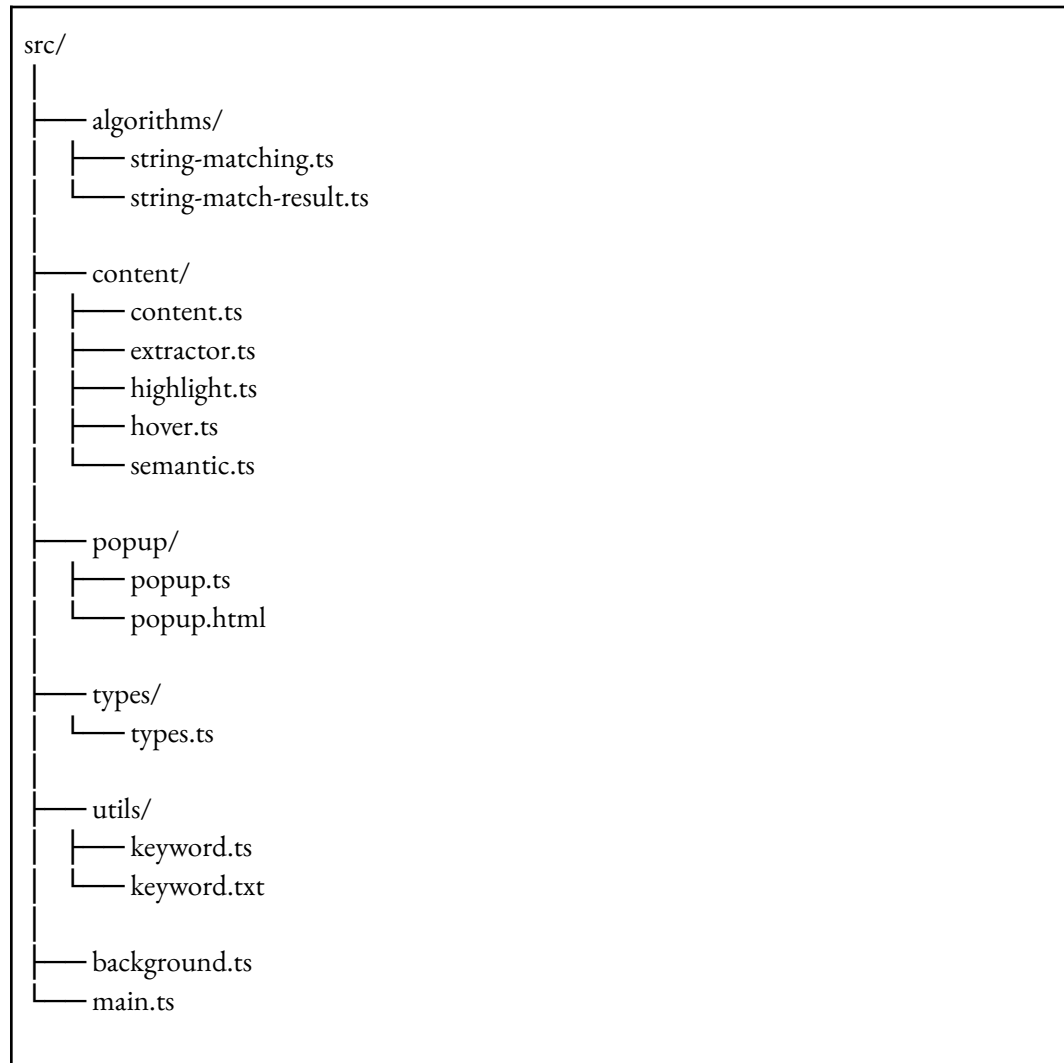
Lapisan *content* menjadi lapisan tempat logika ekstensi berada. Di lapisan ini dapat ditemukan pengestrakan *text node*, pemrosesan *exact*, *regex*, dan *fuzzy matching*, pembuatan statistik *scan*, serta pemberian *custom DOM element* pada konten terdeteksi mengandung unsur judi *online*. Statistik yang dibuat tersebut diberikan kepada lapisan *popup* melalui lapisan *background* untuk ditampilkan di antarmuka untuk dilihat oleh pengguna.

Bab IV: Implementasi dan Pengujian Program

4.1. Implementasi Program

4.1.1. Struktur Folder

Berikut merupakan struktur folder repositori dari program yang dikembangkan oleh tim.



4.1.2. Arsitektur Program

Judol Detector dibangun menggunakan *framework* Vanilla + Typescript karena UI yang akan diimplementasikan cukup sederhana dan lebih fokus terhadap implementasi algoritma. Secara garis besar, tim juga memanfaatkan konsep chromium extension untuk pengembangan proyek. Bagian dibagi menjadi popup, background script, dan content script.

Ketiga komponen ini saling berinteraksi untuk melakukan proses scanning, manipulasi DOM, serta menampilkan hasil deteksi kepada pengguna.

Komponen popup berfungsi sebagai antarmuka pengguna (UI) yang digunakan untuk menampilkan statistik hasil deteksi dan juga menyediakan opsi blur teks. Komponen ini diimplementasikan pada file popup.ts. Background script berfungsi sebagai message hub yang menghubungkan komunikasi antara popup dan content script. Dengan adanya background script, pengiriman event seperti trigger scan atau update statistik dapat dilakukan secara terpusat. Terakhir, komponen utama sistem ini berada pada content script, yang bertugas sebagai entry point seluruh pipeline scanning. Content script melakukan pengambilan text node dari DOM, menjalankan algoritma pattern matching, melakukan highlight pada elemen yang terdeteksi, serta memasang hover listener untuk setiap elemen yang berhasil dideteksi.

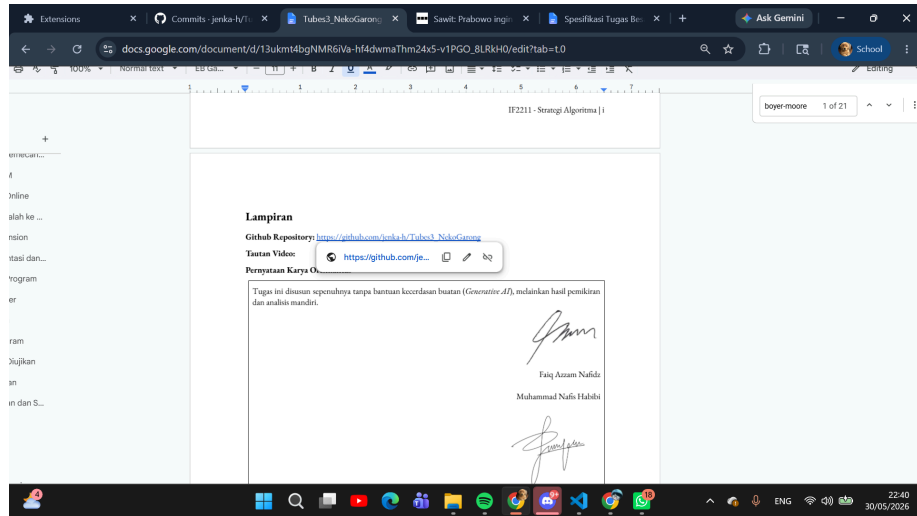
4.1.3. Alur Program

Secara umum, alur program dimulai ketika halaman web selesai dimuat. Content script kemudian akan dijalankan untuk memulai proses scanning terhadap halaman web. Tahap pertama adalah proses ekstraksi text node menggunakan TreeWalker. Sistem mengambil seluruh teks pada halaman web sambil mengabaikan elemen tertentu seperti <script>, <style>, dan node kosong. Setelah seluruh text node diperoleh, sistem menjalankan proses pattern matching pada setiap node menggunakan algoritma yang telah diimplementasikan.

Apabila suatu keyword berhasil dideteksi, sistem akan menyimpan informasi hasil pencarian seperti keyword, posisi match, algoritma yang digunakan, dan waktu eksekusi. Tahap berikutnya adalah proses highlighting. Sistem membangun ulang teks menggunakan DocumentFragment lalu memberikan elemen <mark> pada bagian yang terdeteksi tanpa merusak struktur halaman web. Tim menggunakan <mark> sebagai bentuk konvensi yang sesuai untuk *'highlighted'* element. Setelah proses selesai, sistem memasang hover listener pada elemen HTML dengan semantik representasi terdekat yang bermakna, berisi informasi sesuai permintaan spesifikasi.

Berikut merupakan tahapan yang bisa dilakukan untuk melakukan pengujian program.

1. Lakukan git clone terhadap repositori pada lampiran.



2. Ketik “npm run dev” pada terminal di repositori tersebut.

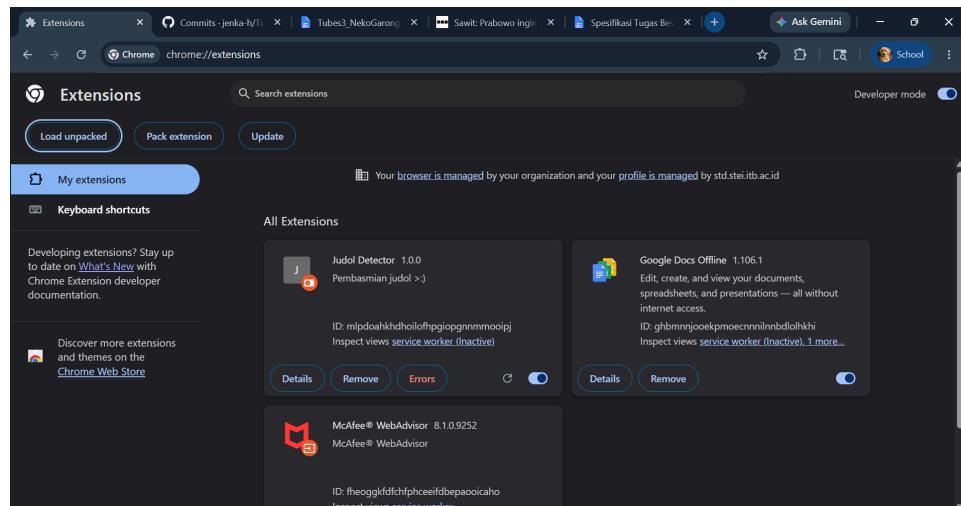
```
PS C:\Gawai\Tubes3_NekoGarong\Tubes3_NekoGarong> npm run dev
Unknown input options: platform. Allowed options: acorn, acornInjectPlugins, cache, context, experimentalCacheExpiry, external, inlineDynamicImports, input, makeAbsoluteExtern
alRelative, manualChunks, maxParallelFileOps, maxParallelFileReads, moduleContext, onwarn, perf, plugins, preserveEntrySignatures, preserveModules, preserveSymlinks, shims,
ingExports, strictDeprecations, treeshake, watch

VITE v8.0.12 ready in 1102 ms

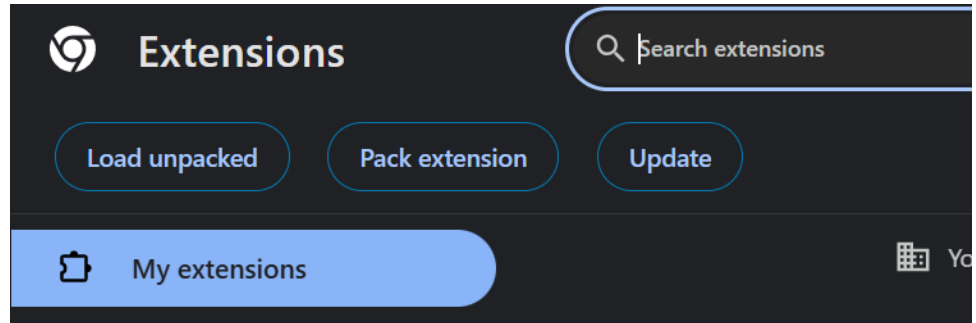
  BROWSER
  EXTENSION
  TOOLS

➔ CR375: Load dist as unpacked extension
```

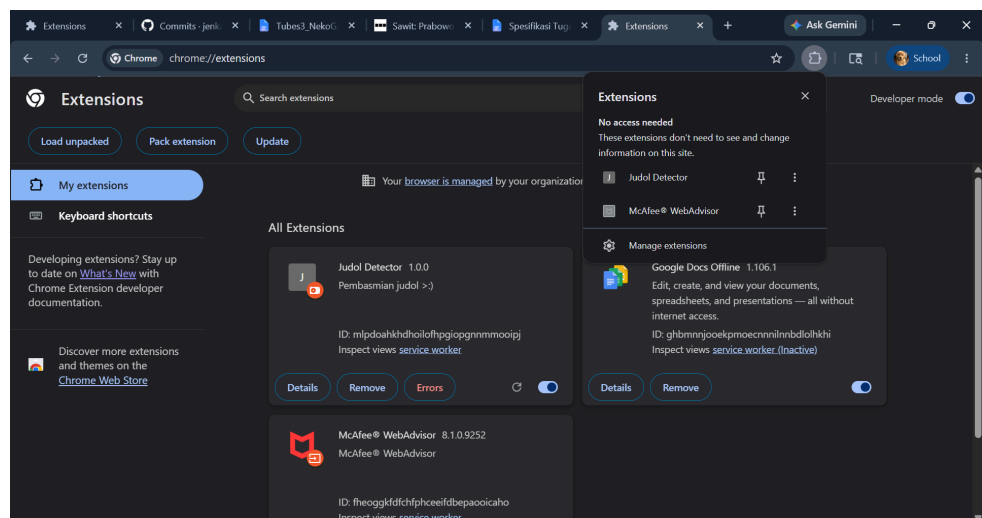
3. Kemudian, pergi ke chrome://extensions dan buka developer mode.



4. Pilih menu “Load unpacked” lalu masukkan folder hasil build ekstensi dist/ pada repositori.



- Setelah ekstensi berhasil dimuat, program siap diuji pada halaman web yang diinginkan.



4.1.4. Struktur Data

Berikut merupakan beberapa struktur data unik.

No.	Struktur Data	Definisi	Atribut
1.	TextNodeData	Struktur data yang digunakan untuk menyimpan informasi node teks beserta teks asli yang dimiliki elemen DOM.	{ node: Text, originalText: string }
2.	ScanStatistic	Struktur data yang menyimpan statistik hasil pemindaian, seperti jumlah kata yang terdeteksi, waktu eksekusi, dan metode yang digunakan.	{ totalMatches, methodResults: Map<string, MethodResult>, topKeywords }
3.	MethodResult	Struktur data yang menyimpan hasil eksekusi suatu algoritma pencarian	{ executionTime, comparisonCount }

		string, termasuk posisi kecocokan keyword.	
4.	ElementMatch Result	Struktur data yang merepresentasikan hasil pencocokan keyword pada suatu elemen HTML tertentu.	{ node: Text, result: StringMatchResult, execTime }
5.	StringMatchResult	Struktur data yang digunakan untuk menyimpan hasil pencarian string, seperti indeks kecocokan dan keyword yang ditemukan.	{ method, input, matchPosition: Map<string, number[]> }
6.	AhoCorasickTrie	Struktur data trie yang digunakan pada algoritma Aho-Corasick untuk melakukan pencarian banyak pola secara efisien.	{ next: Map, link, parent, char, word }
7.	Range	Struktur data yang merepresentasikan rentang indeks, biasanya berupa posisi awal dan akhir suatu kecocokan teks.	{ start: number, end: number, keyword: string }
8.	ImageTarget	Struktur data yang digunakan untuk menyimpan informasi target gambar yang akan diproses atau dianalisis.	{ element: HTMLImageElement, src: string }

Berikut merupakan beberapa fungsi dan prosedur unik.

No	Nama	Deskripsi
1	extractTextNodes()	Menelusuri pohon DOM dan menyaring node script, style, serta node kosong.
2	extractImages()	Mengambil gambar valid dengan tipe MIME yang didukung.
3	recognizeImage(dataUrl, lang)	Melakukan OCR menggunakan Tesseract.js.
4	coverImage(target, src)	Menimpa gambar dengan overlay Mr. Peanut.

5	<code>removeCoverImage()</code>	Menghapus seluruh gambar dengan atribut data-judol-cover.
6	<code>exactMatching(input, method)</code>	Dispatcher untuk algoritma exact matching (KMP/BM/AC/RK/RGX).
7	<code>fuzzyMatching(input)</code>	Melakukan fuzzy matching menggunakan Levenshtein.
8	<code>knuthMorrisPratt(input, pattern)</code>	Algoritma KMP menggunakan border function.
9	<code>boyerMoore(input, pattern)</code>	Algoritma Boyer-Moore menggunakan Last Occurrence Table.
10	<code>ahoCorasick(input, patterns)</code>	Pencarian multi-pattern menggunakan Trie.
11	<code>rabinKarp(input, pattern)</code>	Pencarian berbasis hash dengan rolling hash.
12	<code>regexMatch(input)</code>	Pencocokan pola menggunakan regular expression.
13	<code>levenshteinDistance(input, pattern, threshold)</code>	Menghitung edit distance menggunakan matriks dynamic programming.
14	<code>createHover()</code>	Membuat elemen popup hover.
15	<code>showElementHover(popup, e, matches)</code>	Menampilkan dan mengisi informasi popup hover.
16	<code>applyAllHighlights(matches, blur)</code>	Membungkus teks yang cocok dengan elemen <mark> (sesuai best practice).
17	<code>findSemanticHoverContainer(element)</code>	Menelusuri DOM untuk mencari semantic container.
18	<code>attachSemanticListeners(matches, popup)</code>	Menambahkan event mouse pada semantic container.

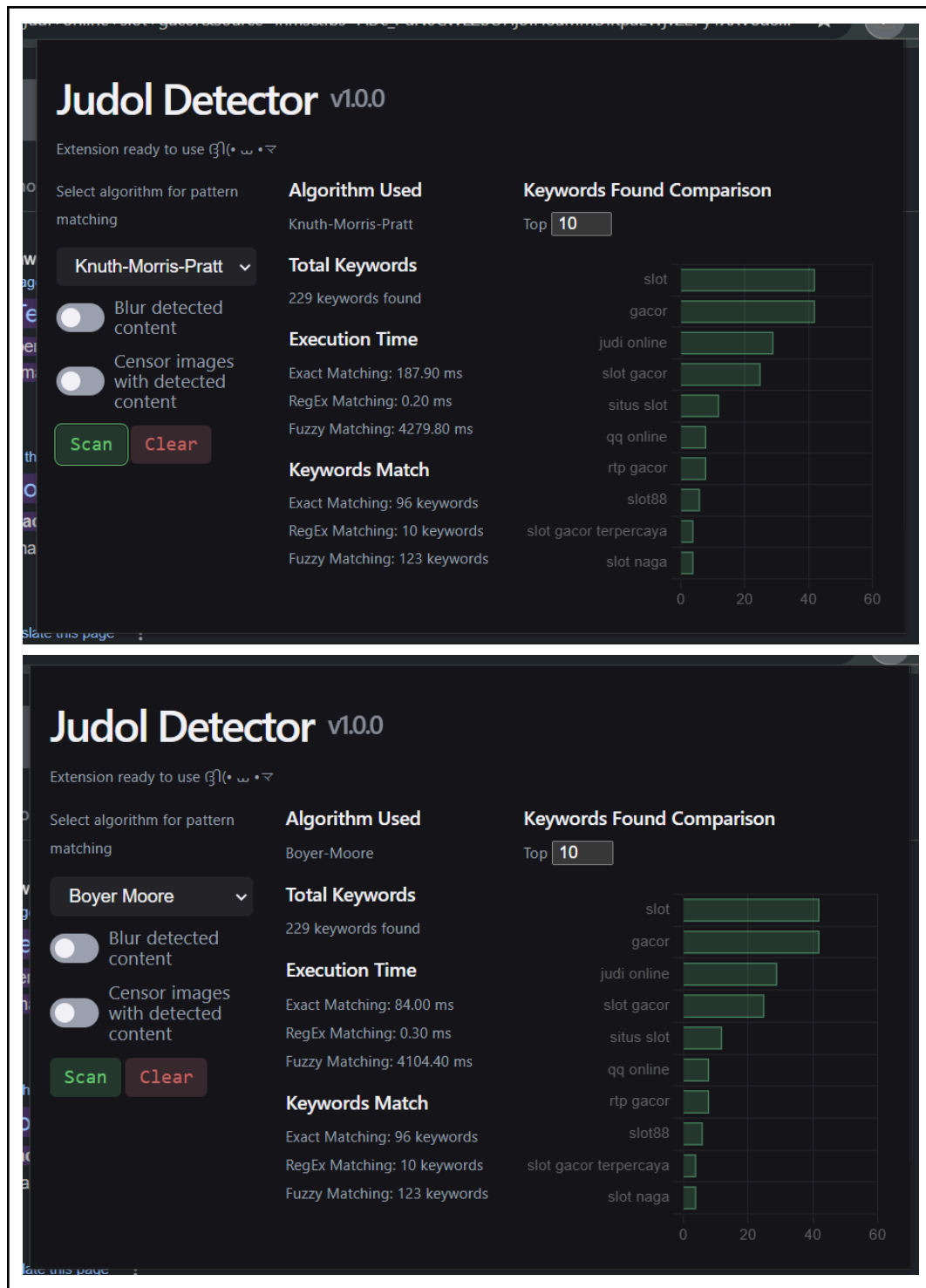
4.2. Pengujian Program

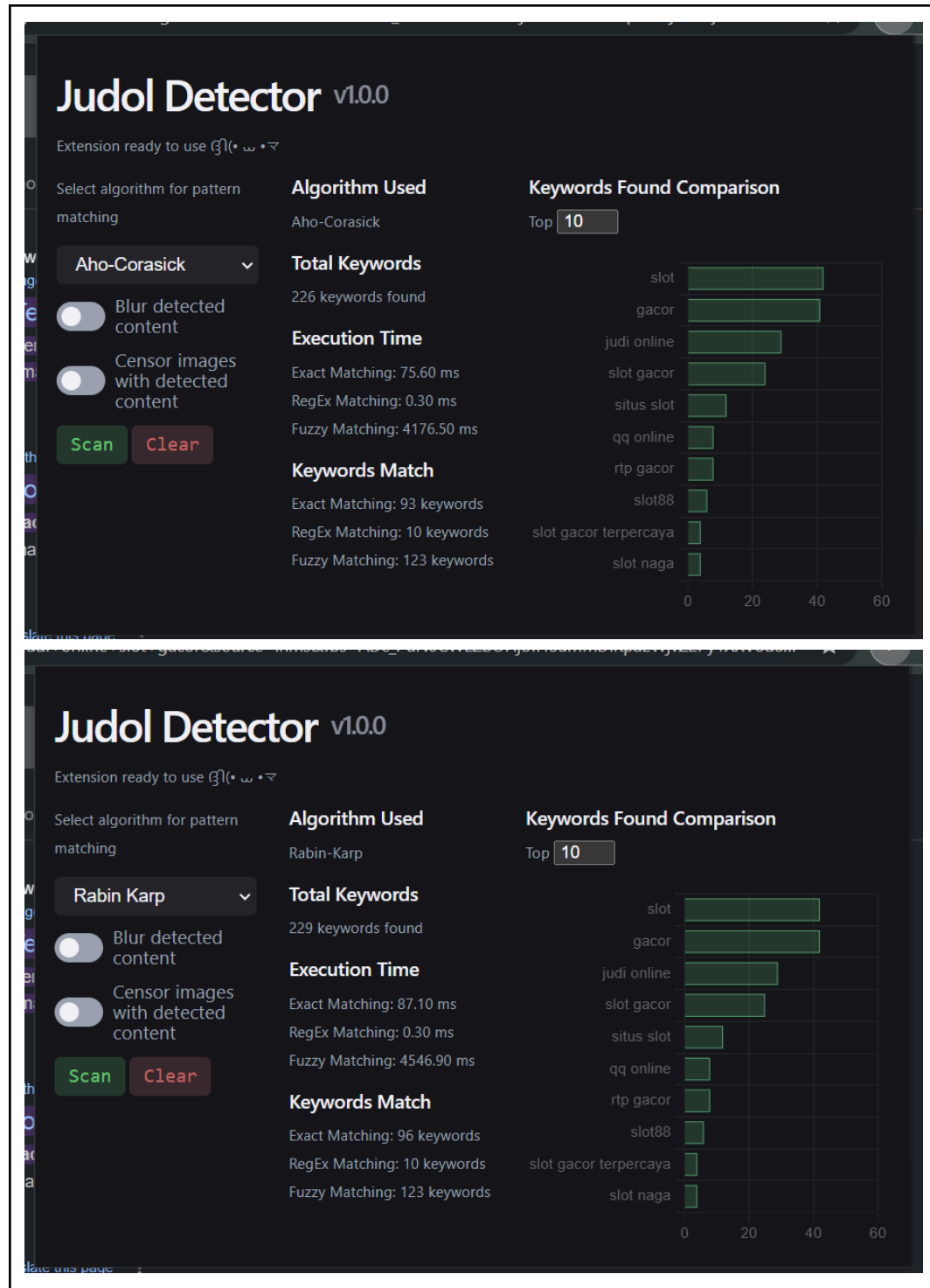
4.2.1. Faktor yang Diujikan

- Kecepatan algoritma (ms)
- Banyak kata terdeteksi

4.2.2. Hasil Pengujian

Kasus Uji 1: Pencarian di google (teks)		
Link: https://www.google.com/search?sca_esv=37037dcbf46c071d&q=judi+online+slot+gacor&source=lnms&fbs=ADc_l-aN0CWEZBOHjofHoaMMDiKpaEWjvZ2Py1XXV8d8Kvli3p-ML-906rRL_m6h4jR-tdAeyw6pOVABma0FfM0NmtARdcuGnZVoiSrEQCt2b10nLmwmT1dasrScerQpt4E6jHtRrpXwxf-1k02VNI6kp7wOxJh_EfwYswWuCm3GjVi_WfrDLBshQc4BbO6d1FgA3tmQQDbam38Aec1F8z7lhhW4AFOb5A&sa=X&ved=2ahUKEwiu9IXyleGUAXVnSmwGHRG5DXsQ0pQJegQIEBAB&biw=1536&bih=730&dpr=1.25		
Algoritma	Kecepatan (ms)	Banyak Kata Terdeteksi
Knuth-Morris-Pratt	187,90	96
Boyer-Moore	84,00	96
Aho-Corasick	75,60	93
Rabin-Karp	87,10	96
Regular Expression	0,20	10
Fuzzy Matching	4279,80	123
Screenshot hasil:		





Hasil pengujian menunjukkan bahwa algoritma Exact Matching mampu mendeteksi banyak kata pada hasil pencarian Google berbasis teks. Aho-Corasick menjadi algoritma exact

matching tercepat, sedangkan Fuzzy Matching mendeteksi kata paling banyak namun membutuhkan waktu eksekusi yang jauh lebih besar.

Kasus Uji 2: Pencarian di google (gambar), untuk OCR

Link:

https://www.google.com/search?sa=X&csca_esv=37037dcbf46c071d&sxsrf=ANbL-n6HQ_Cq7pt9PObaPZAlyALM9k8JcnQ:1780153055796&udm=2&fbs=ADc_l-aN0CWEZBOHjofHoaMMDiKpaEWjvZ2Py1XXV8d8KvII3p-ML-906rRL_m6h4jR-tdAeyw6pOVABma0FfM0NmtARdcuGnZVoiSrEQCt2b10nLmwmT1dasrScerQpt4E6jHtRpXwxf-1k02VNI6kp7wO3C8TkoRLBBZ1LJZoHIeFL1DwSjslhoTqPz9yOA3Wnl8PhLApZPv4OGwQY8aVA2rGR-si2w&q=judi+online+slot+gacor&ved=2ahUKEwijklX0ouGUAXWOzzgGHWnnBYYQtKgLegQIFhAB&biw=1536&bih=730&dpr=1.25

Algoritma	Kecepatan (ms)	Banyak Kata Terdeteksi
Knuth-Morris-Pratt	291,40	360
Boyer-Moore	138,80	360
Aho-Corasick	157,20	327
Rabin-Karp	138,70	360
Regular Expression	0,40	26
Fuzzy Matching	4705,00	441

Screenshot hasil:

Judol Detector v1.0.0

Extension ready to use

Select algorithm for pattern matching

Knuth-Morris-Pratt

☐ Blur detected content

☐ Censor images with detected content

Scan

Clear

Algorithm Used

Knuth-Morris-Pratt

Total Keywords

827 keywords found

Execution Time

Exact Matching: 291.40 ms

RegEx Matching: 0.40 ms

Fuzzy Matching: 4705.00 ms

Keywords Match

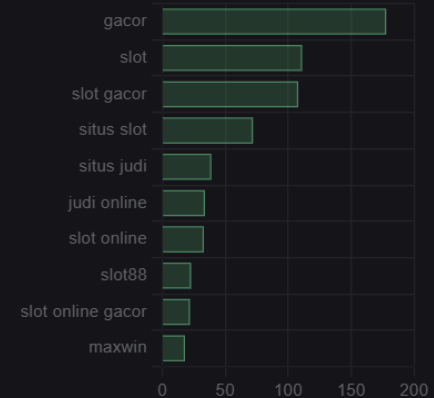
Exact Matching: 360 keywords

RegEx Matching: 26 keywords

Fuzzy Matching: 441 keywords

Keywords Found Comparison

Top 10



Judol Detector v1.0.0

Extension ready to use

Select algorithm for pattern matching

Boyer Moore

☐ Blur detected content

☐ Censor images with detected content

Scan

Clear

Algorithm Used

Boyer-Moore

Total Keywords

827 keywords found

Execution Time

Exact Matching: 138.80 ms

RegEx Matching: 0.30 ms

Fuzzy Matching: 4778.60 ms

Keywords Match

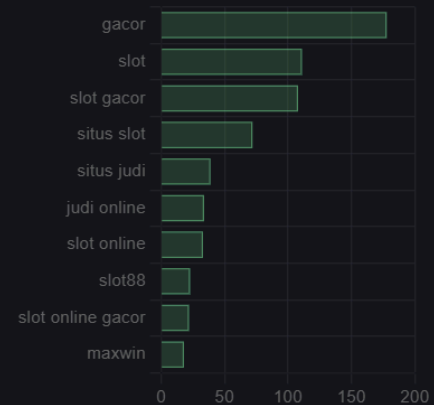
Exact Matching: 360 keywords

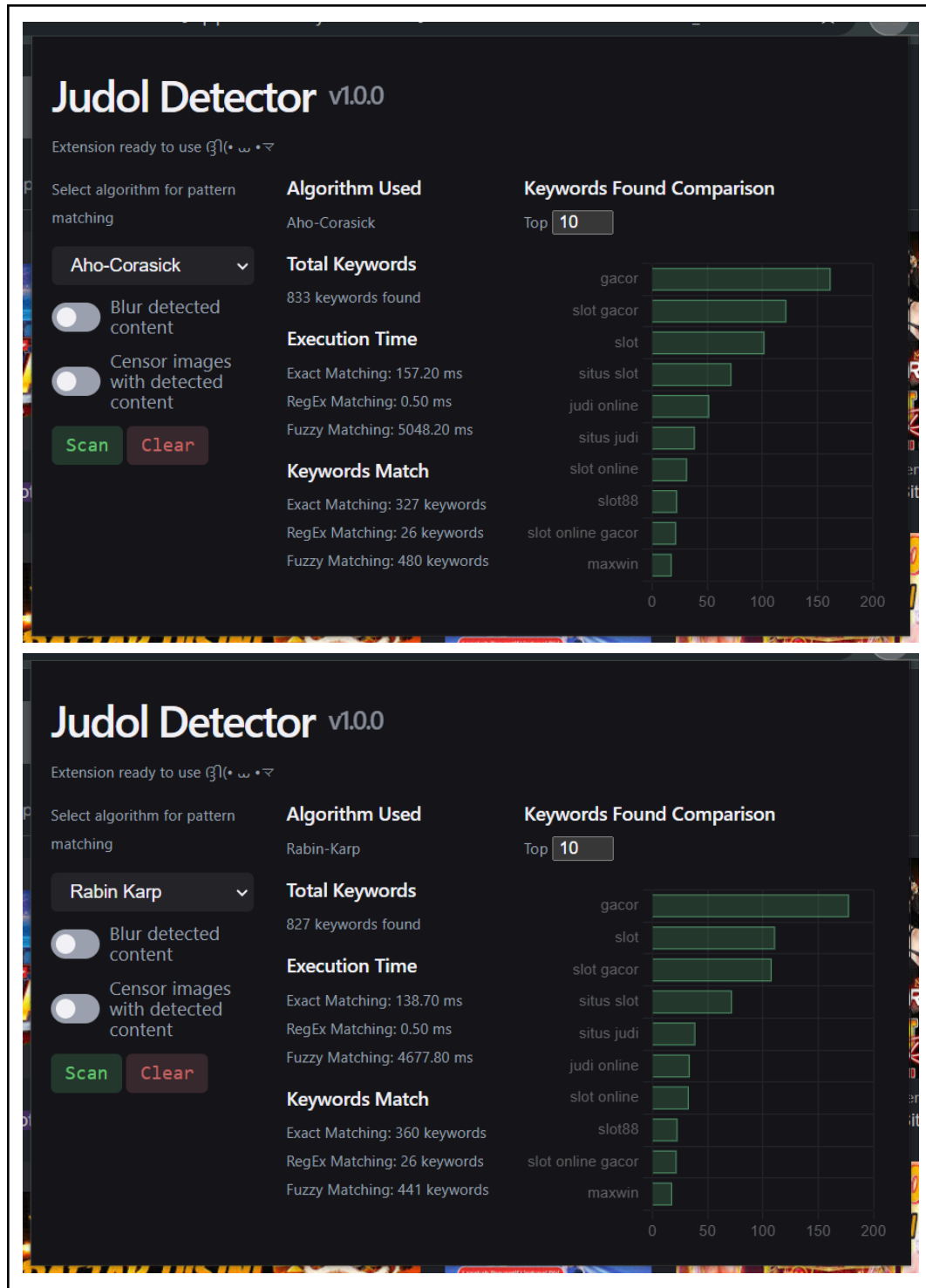
RegEx Matching: 26 keywords

Fuzzy Matching: 441 keywords

Keywords Found Comparison

Top 10





Kasus pengujian kedua dilakukan pada pencarian gambar Google untuk menguji fitur OCR. Hasil pengujian menunjukkan bahwa algoritma mampu mendeteksi lebih banyak

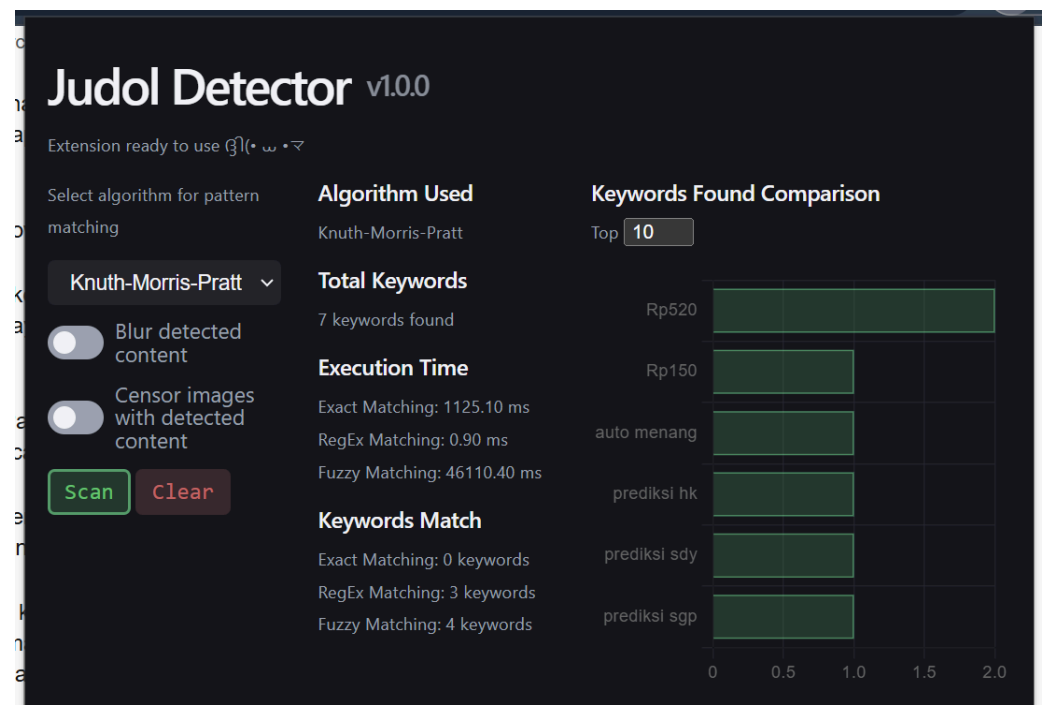
keyword dibandingkan pencarian teks biasa karena teks pada gambar juga berhasil diekstraksi menggunakan OCR.

Kasus Uji 3: Artikel Biasa

Link: <https://www.bbc.com/indonesia/articles/crrk14e41ddo>

Algoritma	Kecepatan (ms)	Banyak Kata Terdeteksi
Knuth-Morris-Pratt	1125,10	0
Boyer-Moore	389,70	0
Aho-Corasick	207,30	0
Rabin-Karp	602,10	0
Regular Expression	± 0,70	3
Fuzzy Matching	± 47000,00	4

Screenshot hasil:



Judol Detector v1.0.0

Extension ready to use

Select algorithm for pattern matching

Boyer Moore

☐ Blur detected content

☐ Censor images with detected content

Scan

Clear

Algorithm Used

Boyer-Moore

Total Keywords

7 keywords found

Execution Time

Exact Matching: 389.70 ms

RegEx Matching: 0.80 ms

Fuzzy Matching: 47011.30 ms

Keywords Match

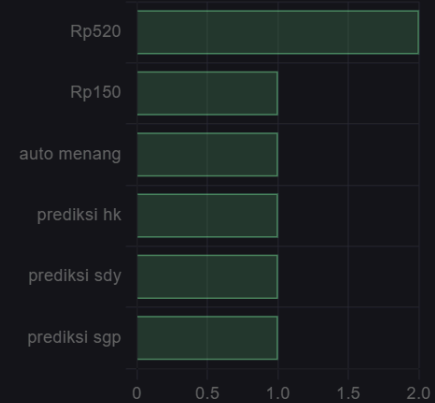
Exact Matching: 0 keywords

RegEx Matching: 3 keywords

Fuzzy Matching: 4 keywords

Keywords Found Comparison

Top 10



Judol Detector v1.0.0

Extension ready to use

Select algorithm for pattern matching

Aho-Corasick

☐ Blur detected content

☐ Censor images with detected content

Scan

Clear

Algorithm Used

Aho-Corasick

Total Keywords

7 keywords found

Execution Time

Exact Matching: 207.30 ms

RegEx Matching: 0.60 ms

Fuzzy Matching: 47855.80 ms

Keywords Match

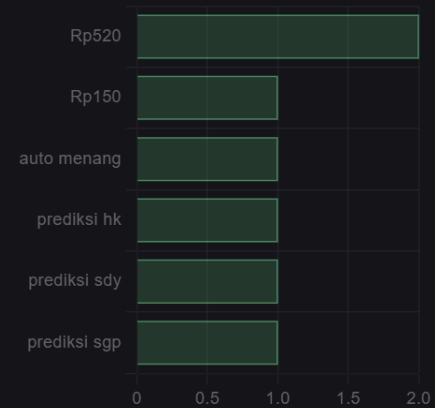
Exact Matching: 0 keywords

RegEx Matching: 3 keywords

Fuzzy Matching: 4 keywords

Keywords Found Comparison

Top 10





Kasus kali ini menggunakan artikel berita biasa yang tidak berkaitan dengan judi. Sebagian besar algoritma exact matching tidak mendeteksi keyword apa pun, menunjukkan bahwa sistem cukup baik dalam menghindari false detection pada artikel normal.

Kasus Uji 4: Akun Instagram Judi Online		
Link: https://www.instagram.com/zeus.gacor.hari.ini/		
Algoritma	Kecepatan (ms)	Banyak Kata Terdeteksi
Knuth-Morris-Pratt	441,70	3
Boyer-Moore	184,30	3
Aho-Corasick	75,00	3
Rabin-Karp	171,80	3
Regular Expression	± 37,5	1
Fuzzy Matching	± 13660	4
Screenshot hasil:		

Judol Detector v1.0.0

Extension ready to use

Select algorithm for pattern matching

Knuth-Morris-Pratt

☐ Blur detected content

☐ Censor images with detected content

Scan

Clear

Algorithm Used

Knuth-Morris-Pratt

Total Keywords

8 keywords found

Execution Time

Exact Matching: 441.70 ms

RegEx Matching: 0.30 ms

Fuzzy Matching: 13905.00 ms

Keywords Match

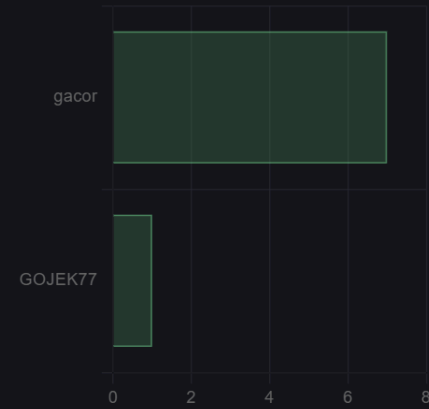
Exact Matching: 3 keywords

RegEx Matching: 1 keywords

Fuzzy Matching: 4 keywords

Keywords Found Comparison

Top 10



Judol Detector v1.0.0

Extension ready to use

Select algorithm for pattern matching

Boyer Moore

☐ Blur detected content

☐ Censor images with detected content

Scan

Clear

Algorithm Used

Boyer-Moore

Total Keywords

8 keywords found

Execution Time

Exact Matching: 184.30 ms

RegEx Matching: 0.80 ms

Fuzzy Matching: 13377.80 ms

Keywords Match

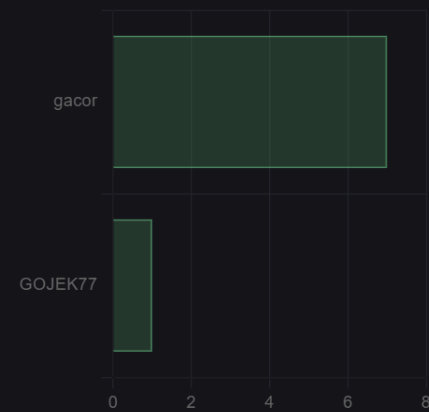
Exact Matching: 3 keywords

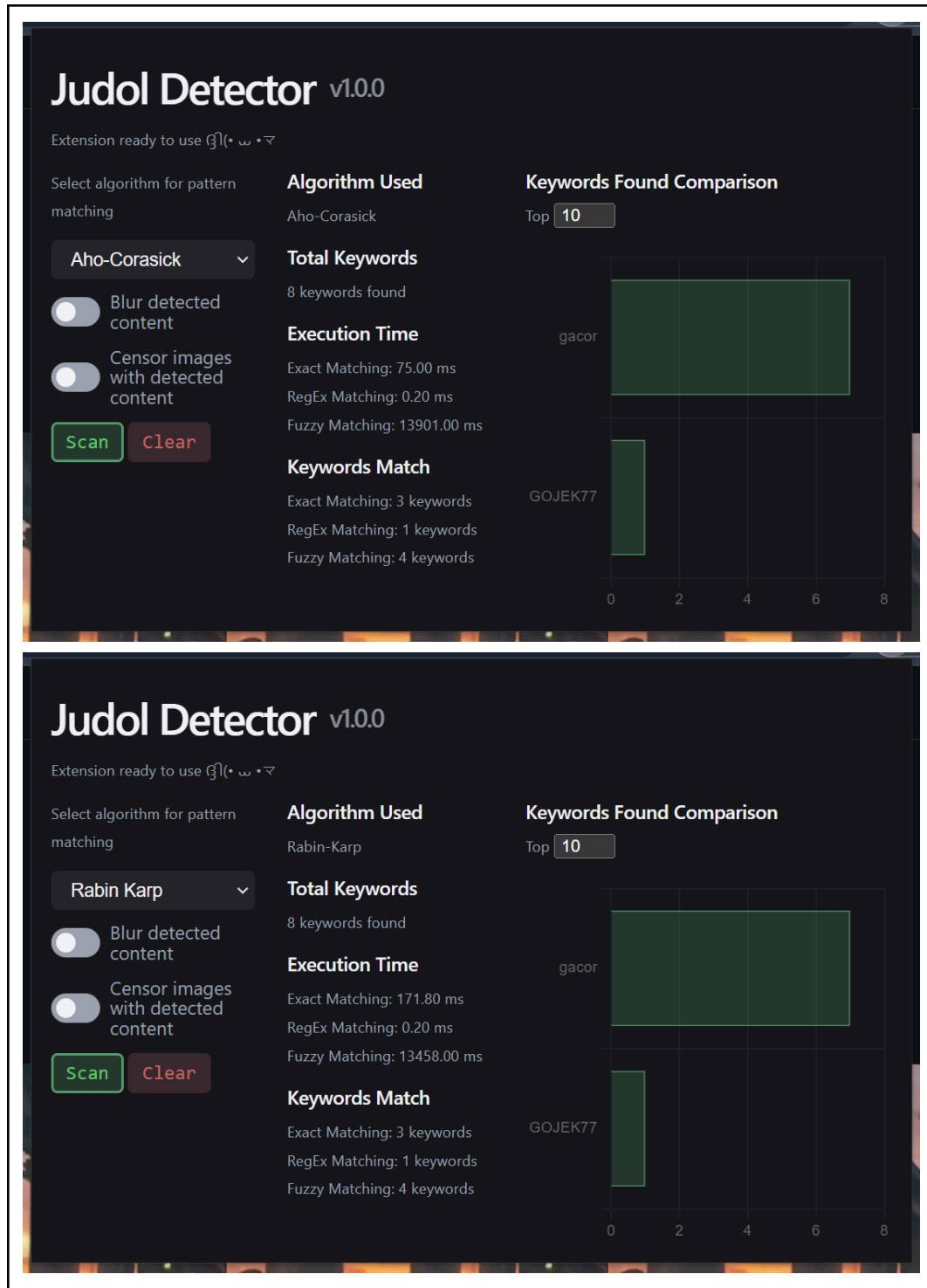
RegEx Matching: 1 keywords

Fuzzy Matching: 4 keywords

Keywords Found Comparison

Top 10



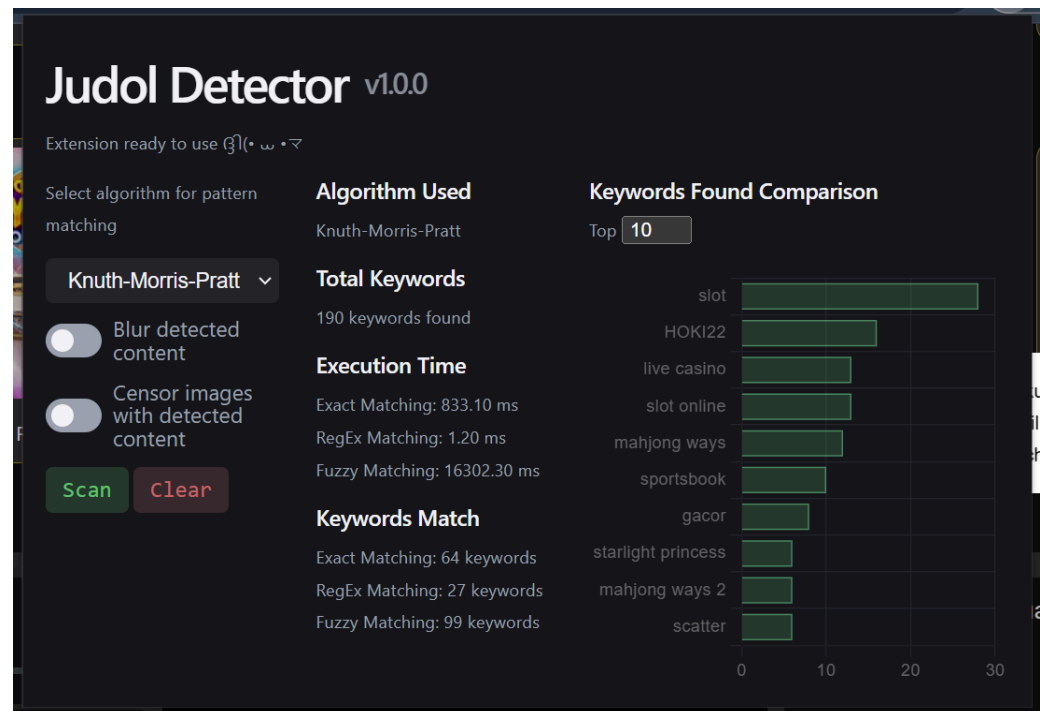


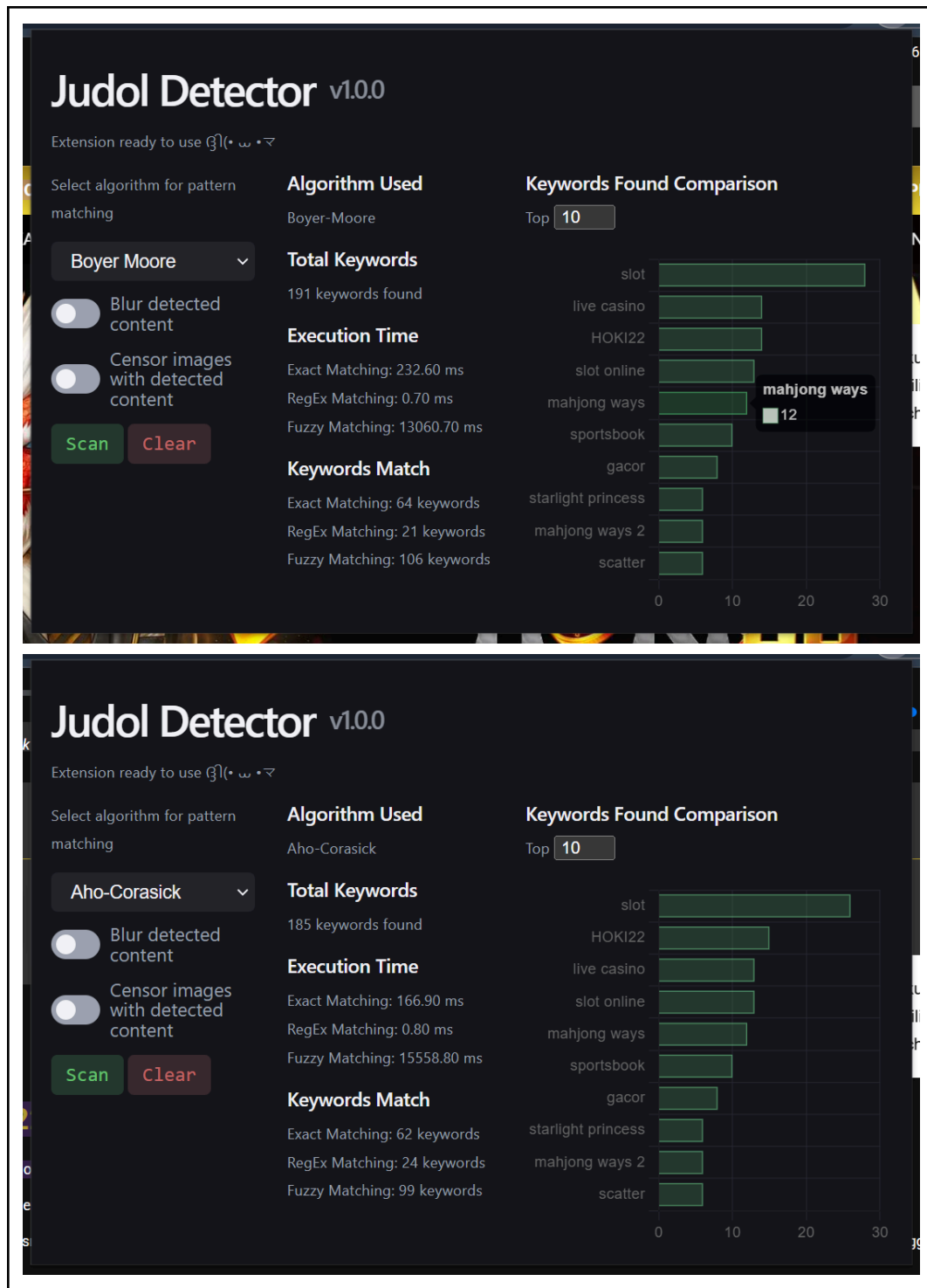
Berdasarkan hasil pengujian, algoritma Aho-Corasick memiliki performa tercepat dalam mendeteksi banyak keyword secara bersamaan dengan waktu eksekusi 75,00 ms dan

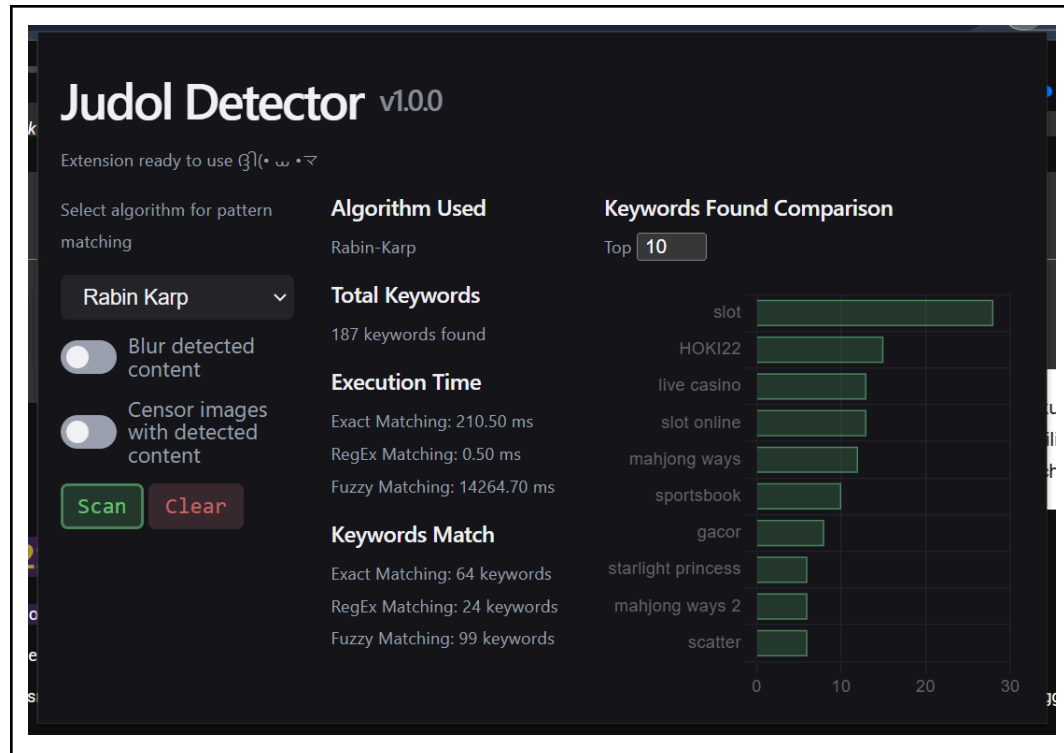
berhasil mendeteksi 3 kata. Boyer-Moore dan Rabin-Karp juga menunjukkan performa yang cukup baik dengan waktu eksekusi di bawah 200 ms.

Kasus Uji 5: Website Judi Online		
Link: https://kopilajang.store/		
Algoritma	Kecepatan (ms)	Banyak Kata Terdeteksi
Knuth-Morris-Pratt	833,10	64
Boyer-Moore	232,60	64
Aho-Corasick	166,90	62
Rabin-Karp	210,50	64
Regular Expression	± 0.75 (Rata-rata)	± 25
Fuzzy Matching	$\pm 14500,00$	99

Screenshot hasil:







Tiap algoritma pada kasus kelima memiliki hasil keyword yang berbeda. Hal ini disebabkan karena laman pada test case bersifat dinamis, sedangkan program yang dibuat saat ini hanya mampu menangani laman statis. Sementara itu, terlihat bahwa Aho-Corasick merupakan algoritma tercepat diantara keempat algoritma yang tergolong dalam Exact Matching. Hasil ini sesuai dengan teori yang telah diberikan.

Analisis:

Pada pengujian ini digunakan beberapa algoritma string matching, yaitu Knuth-Morris-Pratt (KMP), Boyer-Moore (BM), Aho-Corasick (AC), Rabin-Karp (RK), Regular Expression (Regex), dan Fuzzy Matching berbasis Levenshtein Distance. Seluruh algoritma digunakan untuk mendeteksi keyword yang berkaitan dengan judi online pada berbagai jenis laman web, seperti hasil pencarian Google, gambar OCR, artikel biasa, media sosial, hingga website judi online secara langsung.

Algoritma Knuth-Morris-Pratt (KMP) bekerja dengan memanfaatkan border function atau prefix table untuk menghindari perbandingan ulang pada karakter yang telah diperiksa sebelumnya. Pendekatan ini membuat KMP memiliki performa yang stabil pada proses pencarian exact matching. Namun, karena proses traversal karakter dilakukan secara

berurutan, waktu eksekusi KMP masih lebih lambat dibandingkan Boyer-Moore dan Aho-Corasick pada sebagian besar kasus pengujian.

Boyer-Moore (BM) menggunakan pendekatan pencocokan dari kanan ke kiri dengan bantuan last occurrence table. Strategi ini memungkinkan algoritma melewati sejumlah karakter sekaligus ketika ditemukan ketidakcocokan. Akibatnya, Boyer-Moore memiliki performa yang relatif cepat pada pencarian teks biasa dan sering kali lebih unggul dibandingkan KMP maupun Rabin-Karp pada pengujian yang dilakukan.

Aho-Corasick (AC) merupakan algoritma multi-pattern matching yang menggunakan struktur data Trie dan failure link. Berbeda dengan algoritma exact matching lainnya yang memeriksa pola satu per satu, Aho-Corasick mampu melakukan pencarian banyak keyword secara bersamaan dalam satu traversal teks. Hal ini menyebabkan Aho-Corasick menjadi algoritma exact matching tercepat pada sebagian besar kasus uji, terutama ketika jumlah keyword yang dicari cukup banyak.

Rabin-Karp (RK) menggunakan teknik rolling hash untuk mempercepat proses pencocokan string. Algoritma ini bekerja dengan membandingkan nilai hash dari substring teks terhadap hash pattern yang dicari. Pada implementasi ini, Rabin-Karp memiliki performa yang cukup baik dan kompetitif terhadap Boyer-Moore, meskipun tetap dipengaruhi kemungkinan collision hash yang dapat menyebabkan pemeriksaan ulang karakter.

Regular Expression (Regex) bekerja dengan mencocokkan pola berdasarkan expression yang telah didefinisikan sebelumnya. Regex memiliki waktu eksekusi paling cepat pada hampir seluruh pengujian karena engine regex modern telah dioptimalkan secara internal. Namun, kemampuan deteksi Regex terbatas pada pola yang benar-benar sesuai dengan expression yang dibuat, sehingga jumlah keyword yang terdeteksi relatif lebih sedikit dibandingkan algoritma lainnya.

Fuzzy Matching menggunakan algoritma Levenshtein Distance untuk menghitung tingkat kemiripan antar kata. Berbeda dengan algoritma exact matching, fuzzy matching tetap dapat mendeteksi kata yang mengalami modifikasi, typo, maupun variasi penulisan. Hal ini menyebabkan jumlah keyword yang berhasil dideteksi menjadi lebih banyak. Namun, proses perhitungan edit distance membutuhkan kompleksitas komputasi yang tinggi akibat

penggunaan dynamic programming matrix, sehingga waktu eksekusinya jauh lebih lambat dibandingkan algoritma lainnya.

Berdasarkan hasil pengujian, Aho-Corasick memberikan performa terbaik pada kategori exact matching karena mampu melakukan pencarian multi-pattern secara efisien. Sementara itu, Fuzzy Matching memiliki kemampuan deteksi paling tinggi tetapi dengan konsekuensi waktu eksekusi yang sangat besar. Regex menjadi algoritma tercepat, namun fleksibilitas dan cakupan deteksinya relatif terbatas. Perbedaan jumlah keyword antar algoritma juga dipengaruhi oleh sifat laman web yang dinamis, sedangkan program yang dibuat saat ini hanya memproses kondisi DOM statis pada saat scanning dilakukan

Bab V: Kesimpulan dan Saran

5.1. Kesimpulan

Pada Tugas Besar 3 ini, tim berhasil mengimplementasikan chromium browser extension bernama Judol Detector menggunakan bahasa pemrograman TypeScript dengan memanfaatkan algoritma pattern matching untuk mendeteksi konten judi online pada halaman web. Algoritma KMP bekerja secara sistematis dengan memanfaatkan border function untuk menghindari perbandingan karakter yang berulang, sehingga pencarian dapat dilakukan secara linear. Di sisi lain, Boyer-Moore melakukan pencocokan dari kanan ke kiri dan mampu melakukan skipping karakter dalam jumlah besar, sehingga pada beberapa kasus memiliki performa yang lebih cepat dibandingkan KMP. Regular Expression digunakan untuk mendeteksi pola umum berbentuk <kata><angka> yang sering muncul pada konten judi online, sedangkan Weighted Levenshtein Distance digunakan untuk menangani manipulasi karakter visual seperti penggunaan angka atau unicode sebagai pengganti huruf tertentu.

Berdasarkan hasil pengujian, kombinasi exact matching, regular expression, dan fuzzy matching mampu meningkatkan kemampuan sistem dalam mendeteksi variasi keyword judi online. Exact matching sangat efektif untuk mendeteksi keyword identik, sedangkan fuzzy matching membantu mengurangi kemungkinan false negative akibat manipulasi karakter. Regular expression membantu mendeteksi kata-kata yang tidak termasuk dalam keyword, tapi memiliki pola mencurigakan. Namun, penggunaan fuzzy matching dengan threshold yang terlalu rendah dapat meningkatkan false positive sehingga diperlukan penyesuaian parameter yang tepat. Salah satu algoritma yang unggul dalam kecepatan waktu adalah Aho-Corassick, dengan rata-rata merupakan algoritma yang tercepat. Selain itu, algoritma ini lebih lambat dibanding yang lain. Dengan demikian, dapat disimpulkan bahwa pemilihan algoritma sangat mempengaruhi performa dan akurasi sistem deteksi.

5.2. Saran

Untuk pengembangan Source Code lebih lanjut, proses DOM traversal dan preprocessing teks juga dapat dioptimalkan lagi agar extension memiliki performa yang lebih ringan ketika digunakan pada halaman web yang sangat besar. Struktur penyimpanan hasil deteksi dan mekanisme highlight juga

masih dapat dioptimalkan untuk mengurangi penggunaan memori serta mencegah duplicate scanning pada elemen yang sama.

Pada implementasi fuzzy matching, threshold similarity yang digunakan saat ini masih bersifat sedikit statis sehingga masih terdapat kemungkinan false positive maupun false negative pada beberapa kasus tertentu, terutama pada huruf dengan jumlah yang sedikit. Pengembangan lebih lanjut dapat dilakukan menggunakan adaptive threshold, unicode normalization, ataupun preprocessing karakter visual agar sistem lebih akurat dalam menangani manipulasi keyword. Selain itu, fitur OCR menggunakan Tesseract.js untuk mendeteksi teks judi online pada gambar masih dapat dikembangkan lebih lanjut agar sistem tidak hanya bergantung pada text node DOM.

Lampiran

Github Repository: https://github.com/jenka-h/Tubes3_NekoGarong

Tautan Video: <https://youtu.be/8IH8uVsP9k4>

Pernyataan Karya Orisinalitas:

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.



Faiq Azzam Nafidz



Muhammad Nafis Habibi



Jennifer Khang

Tabel Penyelesaian:

No	Poin	Ya	Tidak
1	Extension berhasil di-build dan di-load tanpa kesalahan pada chromium browser dan dikembangkan dengan TypeScript	✓	
2	KMP dan Boyer-Moore diimplementasikan from scratch	✓	
3	Regex handle format <kata><angka> dan berbagai edge case	✓	
4	Pencarian KMP & BM membaca keyword.txt secara iteratif dan tidak menggunakan built-in search function atau library eksternal	✓	
5	Exact matching dan fuzzy matching berjalan benar	✓	
6	Elemen DOM terdeteksi diberi highlight dan terhapus saat rescanning	✓	
7	Tooltip muncul saat hover dengan informasi keyword, algoritma, kemunculan, dan waktu eksekusi	✓	
8	Popup menampilkan statistik realtime (total keyword, perbandingan, waktu eksekusi, jumlah match)	✓	
9	[Bonus] Membuat Video	✓	
10	[Bonus] Implementasi Algoritma Aho-Corasick dan Rabin Karp	✓	
11	[Bonus] Implementasi Censorship / Blur Teks	✓	
12	[Bonus] Implementasi Optical Character Recognition pada Gambar	✓	

Referensi Karakter yang Mirip: <https://util.unicode.org/UnicodeJsps/confusables.jsp>

Daftar Pustaka

- Rinaldi. (2026). Pencocokan String (String/Pattern Matching). Institut Teknologi Bandung.
[https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/23-Pencocokan-string-\(2026\).pdf](https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/23-Pencocokan-string-(2026).pdf)
- Rinaldi. (2026). String Matching dengan Regular Expression. Institut Teknologi Bandung.
[https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/24-String-Matching-dengan-Regex-\(2026\).pdf](https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/24-String-Matching-dengan-Regex-(2026).pdf)
- Amali, F. (2025). Creating a Tool for Converting Images to Text with Tesseract.js. Medium.
<https://medium.com/@fatkhul.amali/create-ocr-application-for-typed-text-use-tesseract-js-2fa2b4e4f6da>
- cp-algorithms. (2025). Aho-Corasick algorithm.
https://cp-algorithms.com/string/aho_corasick.html
- cp-algorithms. (2023). Rabin-Karp algorithm. <https://cp-algorithms.com/string/rabin-karp.html>